DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SEMESTER : IV

COURSE CODE: 22AM402

COURSE NAME: DATA STRUCTURES-II

LIST OF EXPERIMENTS - OUTCOME BASED LAB TASK

FACULTY INCHARGE: MS. D. SUSEELA

MS. I EUGENE BERNA

BANNARI AMMAN INSTITUTE OF TECHNOLOGY
An Autonomous Institution Affiliated to Anna University Chennai - Approved by AICTE - Accredited by NAAC with "A+" Grade
SATHYAMANGALAM - 638 401 ERODE DISTRICT TAMIL NADU INDIA
Ph: 04295-226000 / 221289 Fax: 04295-226666 E-mail: stayahead@bitsathy.ac.in Web: www.bitsathy.ac.in

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Vision of the Department

To achieve excellence in the field of Artificial Intelligence and Machine Learning by focusing on knowledge-centric education systems, integrative partnerships, innovation and cutting-edge research to meet latest industry standards and service the greater cause of society.

Mission of the Department

M1. To develop professionals skilled in the field of Artificial Intelligence and Machine Learning.

M2. To impart quality and value based education and contribute towards the innovation of computing, expert systems, AI, ML to solve complex problems in research and society

Program Outcomes (POs)

Engineering Graduates will be able to:

PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSOs)

Engineering Graduates will be able to:

PSO1. Develop models in Data Science, Machine learning, deep learning and Big data technologies, using AI and modern tools.

PSO2. Formulate solutions for interdisciplinary AI problems through acquired programming knowledge in the respective domains fulfilling with real-time constraints.

BANNARI AMMAN INSTITUTE OF TECHNOLOGY
An Autonomous Institution Affiliated to Anna University Chennai - Approved by AICTE - Accredited by NAAC with "A+" Grade
SATHYAMANGALAM - 638 401 ERODE DISTRICT TAMIL NADU INDIA
Ph: 04295-226000 / 221289 Fax: 04295-226666 E-mail: stayahead@bitsathy.ac.in Web: www.bitsathy.ac.in

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SEMESTER : IV

COURSE CODE: 22AM402

COURSE NAME: DATA STRUCTURES-II

LIST OF EXPERIMENTS - OUTCOME BASED LAB TASK

S. No.	Name of the Experiment
1	Implement a trie data structure to efficiently support autocomplete suggestions based on user input in google docs.
2	Implement an Algorithm to find the shortest route and travel time between two locations within a city's transportation network.
3	Design a cost-efficient telecommunication network to connect multiple cities using Kruskal's algorithm. You need to determine the optimal set of connections (edges) that connect all cities (vertices) while minimizing the cost.
4	Implement a chess game application using backtracking.
5	Implementing Segment Tree for Range Sum Query in a Real-time Data Analytics Platform for student management system.
6	Implement a geographic information system (GIS) for locating a city as node using quadtree

Experiment 1

Implement a trie data structure to efficiently support autocomplete suggestions based on user input in google docs.

Aim:

To implement a trie data structure to efficiently support autocomplete suggestions based on user input.

Algorithm:

1. Initialize Trie

- Create a root node which acts as the starting point of the trie.

2. Insert a word into the Trie

- Start from the root node.
- For each character in the word:
    - If the character is not present in the current node's children:
        - Add the character to the children with a new TrieNode.
    - Move to the child node of the character.
- After the last character, mark isEndOfWord as True.

3. Search for a Prefix

- Start from the root node.
- For each character in the prefix:
    - Check if the character exists in the current node's children.
    - If not found, return False (or Null).
    - Move to the child node of the character.
- Return the node if the end of the prefix is reached.

4. Collect Words Starting from a Node

- If the current node marks the end of a word, add the prefix to the result list.
- For each child node:
    - Recursively collect words starting from the child node, appending the current character to the prefix.

5. Autocomplete Function

- Use the Search for a Prefix function to find the node corresponding to the last character of the input prefix.
- If the prefix node is found, use the Collect Words function to gather all words beginning with the prefix.
- Return the list of words.

Program:

```
class TrieNode:
        def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
        def __init__(self):
        self.root = TrieNode()

        def insert(self, word):
        node = self.root
        for char in word:
        if char not in node.children:
                node.children[char] = TrieNode()
        node = node.children[char]
        node.is_end_of_word = True

        def search(self, prefix):
        node = self.root
          for char in prefix:
        if char not in node.children:
                return []
        node = node.children[char]
        return self._find_words_from_node(node, prefix)

        def _find_words_from_node(self, node, prefix):
        words = []
        if node.is_end_of_word:
        words.append(prefix)

        for char, next_node in node.children.items():
        words.extend(self._find_words_from_node(next_node, prefix + char))

        return words
```

```
def main():
    trie = Trie()

    while True:
    print("Choose an option:")
    print("1. Insert word")
    print("2. Get autocomplete suggestions")
    print("3. Exit")

    choice = input("Enter your choice: ")

    if choice == '1':
    word = input("Enter the word to insert: ").strip()
    trie.insert(word)
    print(f"Inserted '{word}' into the trie.\n")

    elif choice == '2':
    prefix = input("Enter the prefix to search: ").strip()
    suggestions = trie.search(prefix)
    if suggestions:
        print(f"Autocomplete suggestions for '{prefix}': {suggestions}\n")
    else:
        print(f"No suggestions found for '{prefix}'.\n")

    elif choice == '3':
    print("Exiting...")
    break

    else:
    print("Invalid choice. Please choose again.\n")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
Output
Choose an option:
1. Insert word
2. Get autocomplete suggestions
3. Exit
Enter your choice: 1
Enter the word to insert: apple
Inserted 'apple' into the trie.

Choose an option:
1. Insert word
2. Get autocomplete suggestions
3. Exit
Enter your choice: 1
Enter the word to insert: application
Inserted 'application' into the trie.

Choose an option:
1. Insert word
2. Get autocomplete suggestions
3. Exit
Enter your choice: apricot
Invalid choice. Please choose again.

Choose an option:
1. Insert word
2. Get autocomplete suggestions
3. Exit
Enter your choice: 2
Enter the prefix to search: ap
Autocomplete suggestions for 'ap': ['apple', 'application']

Choose an option:
1. Insert word
2. Get autocomplete suggestions
3. Exit
Enter your choice: 3
Exiting...
```

Result:

Thus the program has been implemented Successfully.

Experiment 2

Implement an Algorithm to find the shortest route and travel time between two locations within a city's transportation network.

Aim:

To implement an Algorithm to find the shortest route and travel time between two locations within a city's transportation network.

Algorithm:

1. Initialize Distances and Predecessors:
    - Create a dictionary dist where each node has an initial distance value of infinity (float('infinity')), except for the source node which has a distance of 0.
    - Create a dictionary prev to store the predecessor of each node along the path. Initially, all values are set to None.
2. Initialize Priority Queue:
    - Create a priority queue Q that will store all nodes of the graph. The queue prioritizes nodes based on their distance values in dist.
3. Process Each Node in the Queue:
    - While the queue Q is not empty:
        - Remove the node u from Q that has the smallest distance value.
        - If u is the target node, exit the loop since the shortest path to the target has been found.
        - For each neighbor v of u:
            - Calculate the alternative distance alt as the sum of dist[u] (distance to reach u) and the weight of the edge from u to v (G.length(u, v)).
            - If alt is less than dist[v]:
                - Update dist[v] with the smaller distance alt.
                - Set prev[v] to u to mark u as the predecessor of v on the path.
                - Update the priority of v in the queue Q to reflect the new distance.
4. Reconstruct the Shortest Path:
    - Initialize an empty list path.
    - Start from the target node and traverse backwards using the prev dictionary to reconstruct the path up to the source.
    - Add each node to the beginning of the path list as you move from target to source.
5. Output the Results:
    - Return the path list which now contains the nodes from source to target forming the shortest route.
    - Return the distance value dist[target] which represents the total travel time or distance from source to target.

Program:

```
import heapq

class Graph:
	def __init__(self):
	self.nodes = set()
	self.edges = {}

	def add_node(self, value):
	self.nodes.add(value)
	self.edges[value] = []

	def add_edge(self, from_node, to_node, travel_time):
	self.edges[from_node].append((to_node, travel_time))
	self.edges[to_node].append((from_node, travel_time))  # Assuming undirected graph

def dijkstra(graph, start):
	queue = []
	heapq.heappush(queue, (0, start))
	distances = {node: float('infinity') for node in graph.nodes}
	distances[start] = 0
	shortest_path = {node: None for node in graph.nodes}

	while queue:
	current_distance, current_node = heapq.heappop(queue)

	if current_distance > distances[current_node]:
	continue

	for neighbor, weight in graph.edges[current_node]:
	distance = current_distance + weight

	if distance < distances[neighbor]:
		distances[neighbor] = distance
		shortest_path[neighbor] = current_node
		heapq.heappush(queue, (distance, neighbor))

	return distances, shortest_path

def shortest_route(graph, start, end):
	distances, shortest_path = dijkstra(graph, start)
	route = []
	current_node = end

	while current_node is not None:
	route.append(current_node)
```

```
        current_node = shortest_path[current_node]

        route = route[::-1]

        return route, distances[end]

def main():
        graph = Graph()

        # Add nodes and edges based on user input
        num_nodes = int(input("Enter the number of locations: "))
        print("Enter the locations (one per line):")
        for _ in range(num_nodes):
        node = input().strip()
        graph.add_node(node)

        num_edges = int(input("Enter the number of paths between locations: "))
        print("Enter the paths (format: from_node to_node travel_time):")
        for _ in range(num_edges):
        from_node, to_node, travel_time = input().strip().split()
        graph.add_edge(from_node, to_node, int(travel_time))

        start = input("Enter the starting location: ").strip()
        end = input("Enter the destination location: ").strip()

        route, travel_time = shortest_route(graph, start, end)
        route_str = ' -> '.join(route)

        print(f"Shortest route: {route_str}")
        print(f"Travel time: {travel_time}")

if __name__ == "__main__":
        main()
```

OUTPUT

```
Output
Enter the number of locations: 5
Enter the locations (one per line):
A
B
C
D
E
Enter the number of paths between locations: 6
Enter the paths (format: from_node to_node travel_time):
A B 1
A C 4
B C 2
B D 5
C D 1
D E 3
Enter the starting location: A
Enter the destination location: E
Shortest route: A -> B -> C -> D -> E
Travel time: 7

=== Code Execution Successful ===
```

Result:

Thus the program has been completed successfully.

Experiment 3

Design a cost-efficient telecommunication network to connect multiple cities using Kruskal's algorithm. You need to determine the optimal set of connections (edges) that connect all cities (vertices) while minimizing the cost.

Aim:

To design a cost-efficient telecommunication network to connect multiple cities using Kruskal's algorithm. You need to determine the optimal set of connections (edges) that connect all cities (vertices) while minimizing the cost.

Algorithm:

1. Initialization (__init__):
    - For each vertex, make it its own parent (initialize parent pointers to themselves).
    - Set rank of each vertex to 0.
2. Find (find):
    - For a given vertex, find the root of the set it belongs to.
    - Implement path compression: During the find, point each vertex directly to the root to flatten the structure, which speeds up future operations.
3. Union (union):
    - Connect two elements. If they are in different sets, unify the sets.
    - Use union by rank: Attach the tree with a smaller rank under the root of the tree with a larger rank to keep the tree shallow.
4. Initialize MST and Disjoint Set:
    - Create an empty list mst to store the edges of the MST.
    - Initialize a DisjointSet with the given vertices.
5. Sort Edges by Weight:
    - Sort all the edges in non-decreasing order of their weight.
6. Process Each Edge:
    - For each edge in the sorted list:
        i. Check if the end vertices of the edge are in different sets using the find method.
        ii. If they are in different sets, it means adding this edge does not form a cycle:
            1. Add the edge to the MST.
            2. Use the union method to join the sets of the two vertices.
7. Return the MST:
    - Once all edges are processed, return the list mst containing the edges of the MST.

Usage in Main Function:

1. Input vertices and edges from the user:
    - Ask the user to input vertices and the number of edges.
    - For each edge, get the input in the format city1 city2 cost and store it as tuples in a list.
2. Call the Kruskal's function:
    - Pass the list of vertices and edges to the kruskal function and get the MST.
3. Output the MST:

    - Print the edges in the MST along with their weights.

Program:

```
class DisjointSet:
        def __init__(self, vertices):
        self.parent = {v: v for v in vertices}
        self.rank = {v: 0 for v in vertices}

        def find(self, vertex):
        if self.parent[vertex] != vertex:
        self.parent[vertex] = self.find(self.parent[vertex])  # Path compression
        return self.parent[vertex]

        def union(self, u, v):
        root_u = self.find(u)
        root_v = self.find(v)

          if root_u != root_v:
        # Union by rank
        if self.rank[root_u] > self.rank[root_v]:
                self.parent[root_v] = root_u
        elif self.rank[root_u] < self.rank[root_v]:
                self.parent[root_u] = root_v
        else:
                self.parent[root_v] = root_u
                self.rank[root_u] += 1

def kruskal(vertices, edges):
        mst = []
        disjoint_set = DisjointSet(vertices)

        # Sort edges by weight
        edges = sorted(edges, key=lambda x: x[2])
```

```
    for edge in edges:
    u, v, weight = edge
    if disjoint_set.find(u) != disjoint_set.find(v):
    disjoint_set.union(u, v)
    mst.append(edge)

    return mst

def main():
    vertices = input("Enter the cities (separated by spaces): ").split()
    num_edges = int(input("Enter the number of possible connections: "))

    edges = []
    print("Enter each connection in the format: city1 city2 cost")
    for _ in range(num_edges):
    u, v, weight = input().split()
      edges.append((u, v, int(weight)))

    mst = kruskal(vertices, edges)

    print("The edges in the Minimum Spanning Tree (MST) are:")
    for u, v, weight in mst:
    print(f"{u} - {v}: {weight}")

if __name__ == "__main__":
    main()
```

OUTPUT

```
Output

Enter the cities (separated by spaces): A B C D E
Enter the number of possible connections: 7
Enter each connection in the format: city1 city2 cost
A B 1
A C 2
B C 2
B D 5
C D 1
D E 3
C E 6
The edges in the Minimum Spanning Tree (MST) are:
A - B: 1
C - D: 1
A - C: 2
D - E: 3

=== Code Execution Successful ===
```

Result:

Thus the Program executed Successfully.

Experiment 4

Implement a chess game application using backtracking.

Aim:

To Implement a chess game application using backtracking.

Algorithm:

1. Initialization (__init__):
    - Initialize the chess board with starting positions of pieces.
    - Set white_turn **to True, indicating that white moves first.**
2. **Print the Board (**print_board):
    - Iterate over each row of the board and print the contents, which represent the current state of the game.
3. Get Pawn Moves (get_pawn_moves):
    - Compute possible moves for a pawn located at a given position:
        - Move forward one square if it's empty.
        - Move forward two squares from the starting position if both are empty.
        - Capture diagonally if there's an opponent's piece.
4. Get Moves for Other Pieces (get_rook_moves, get_knight_moves, etc.):
    - These methods are placeholders and should be implemented to return valid moves for other types of pieces based on chess rules.
5. Get Piece Moves (get_piece_moves):
    - Given a piece's position, determine its type and call the respective method to get possible moves.
6. Validate Move (is_valid_move):
    - Check if a move is within board boundaries.
    - Ensure the destination square is either empty or contains an opponent's piece.
    - Verify the move is one of the legal moves for the piece.
7. Make a Move (make_move):
    - If the move is valid, move the piece on the board.
    - Switch the turn to the other player.
8. Play the Game (play_game):
    - Continuously display the board and prompt the current player to make a move.
    - Convert input from algebraic notation (like 'a2') to board indices.
    - Validate and make the move if valid; otherwise, prompt again.
    - This loop continues until conditions for checkmate or stalemate are met (these conditions are not implemented in the provided code).

Program:

```
class ChessGame:
	def __init__(self):
	self.board = [
	['r', 'n', 'b', 'q', 'k', 'b', 'n', 'r'],
	['p', 'p', 'p', 'p', 'p', 'p', 'p', 'p'],
	[' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
	[' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
	[' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
	[' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
	['P', 'P', 'P', 'P', 'P', 'P', 'P', 'P'],
	['R', 'N', 'B', 'Q', 'K', 'B', 'N', 'R']
	]
	self.white_turn = True

	def print_board(self):
	for row in self.board:
	print(' '.join(row))
	print()

	def get_pawn_moves(self, row, col):
	moves = []
	direction = -1 if self.board[row][col].isupper() else 1
	start_row = 6 if self.board[row][col].isupper() else 1

	# Move forward one square
	if self.board[row + direction][col] == ' ':
	moves.append((row + direction, col))

	# Move forward two squares from starting position
	if row == start_row and self.board[row + 2 * direction][col] == ' ':
		moves.append((row + 2 * direction, col))

	# Capture diagonally
	if col - 1 >= 0 and self.board[row + direction][col - 1] != ' ' and self.board[row +
direction][col - 1].isupper() != self.white_turn:
	moves.append((row + direction, col - 1))
	if col + 1 < 8 and self.board[row + direction][col + 1] != ' ' and self.board[row +
direction][col + 1].isupper() != self.white_turn:
	moves.append((row + direction, col + 1))

	return moves

	def get_rook_moves(self, row, col):
	# Placeholder for rook moves
	return []
```

```
def get_knight_moves(self, row, col):
# Placeholder for knight moves
return []

def get_bishop_moves(self, row, col):
# Placeholder for bishop moves
return []

def get_queen_moves(self, row, col):
# Placeholder for queen moves
return []

def get_king_moves(self, row, col):
# Placeholder for king moves
return []

def get_piece_moves(self, row, col):
piece = self.board[row][col].lower()
if piece == 'p':
return self.get_pawn_moves(row, col)
elif piece == 'r':
return self.get_rook_moves(row, col)
elif piece == 'n':
return self.get_knight_moves(row, col)
elif piece == 'b':
return self.get_bishop_moves(row, col)
elif piece == 'q':
return self.get_queen_moves(row, col)
elif piece == 'k':
return self.get_king_moves(row, col)
else:
return []

def is_valid_move(self, row, col, new_row, new_col):
# Check if the move is within the board boundaries
if new_row < 0 or new_row >= 8 or new_col < 0 or new_col >= 8:
return False

# Check if the destination square is empty or contains an opponent's piece
if self.board[new_row][new_col].isupper() == self.white_turn:
return False

# Check if the move is legal for the piece
piece_moves = self.get_piece_moves(row, col)
for move in piece_moves:
```

```
        if move == (new_row, new_col):
            return True

        return False

        def make_move(self, row, col, new_row, new_col):
        # Move the piece
        piece = self.board[row][col]
        self.board[row][col] = ' '
        self.board[new_row][new_col] = piece

        # Switch the turn
        self.white_turn = not self.white_turn

        def play_game(self):
        while True:
        self.print_board()
        print("White's turn" if self.white_turn else "Black's turn")

        # Get user input for the move
        source = input("Enter the source square (e.g., a2): ")
        destination = input("Enter the destination square (e.g., a4): ")

        # Convert input to row and column indices
        source_col = ord(source[0]) - ord('a')
        source_row = 8 - int(source[1])
        dest_col = ord(destination[0]) - ord('a')
        dest_row = 8 - int(destination[1])

        # Check if the move is valid
        if self.is_valid_move(source_row, source_col, dest_row, dest_col):
            self.make_move(source_row, source_col, dest_row, dest_col)
        else:
            print("Invalid move. Please try again.")
            continue

        # Check for checkmate or stalemate
        # (implementation omitted for brevity)

game = ChessGame()
game.play_game()
```

OUTPUT

```
Output
r n b q k b n r
p p p p p p p p




P P P P P P P P
R N B Q K B N R

White's turn
Enter the source square (e.g., a2): a2
Enter the destination square (e.g., a4): a4
r n b q k b n r
p p p p p p p p


P

  P P P P P P P
R N B Q K B N R

Black's turn
Enter the source square (e.g., a2):
```

Result:

Thus the program has been implemented successfully.

Experiment 5
Implementing Segment Tree for Range Sum Query in a Real-time Data Analytics Platform for student management system.

Aim:
To implement a Segment Tree for Range Sum Query in a Real-time Data Analytics Platform for student management system.
Algorithm:

1. Initialization (__init__):
    - Initialize the segment tree with a size of 2n where n is the length of the input array.
    - Construct the segment tree using the construct_segment_tree method.
2. Construct Segment Tree (construct_segment_tree):
    - Recursively build the segment tree:
        - If start equals end, set the tree node to the corresponding array value.
        - Otherwise, recursively build the left and right children and set the current tree node to the sum of these children.
3. Query Range Sum (query_range_sum):
    - Perform a range sum query using the helper method _query_range_sum.
    - This method checks for complete overlap, no overlap, or partial overlap of the range with the current segment and calculates the sum accordingly.
4. Update Value (update_value):
    - Update a value in the input array at a specified index.
    - Calculate the difference between the new value and the current value in the tree.
    - Use _update_value to recursively propagate this difference up the segment tree.
5. Recursive Update Value (_update_value):
    - If the index is within the current segment, update the current tree node.

    - Recursively update the left and right children segments if the current segment is not a leaf.

Program:
```
class SegmentTree:
        def __init__(self, arr):
        self.n = len(arr)
        self.tree = [0] * (2 * self.n)
        self.construct_segment_tree(arr, 0, self.n - 1, 0)

        def construct_segment_tree(self, arr, start, end, index):
        if start == end:
        self.tree[index] = arr[start]
        return arr[start]

        mid = (start + end) // 2
        self.tree[index] = self.construct_segment_tree(arr, start, mid, 2 * index + 1) + \
```

```
            self.construct_segment_tree(arr, mid + 1, end, 2 * index + 2)
        return self.tree[index]

        def query_range_sum(self, query_start, query_end):
        return self._query_range_sum(0, self.n - 1, query_start, query_end, 0)

        def _query_range_sum(self, start, end, query_start, query_end, index):
        if query_start <= start and query_end >= end:  # Complete overlap
        return self.tree[index]
        if query_end < start or query_start > end:      # No overlap
        return 0

        mid = (start + end) // 2
        return self._query_range_sum(start, mid, query_start, query_end, 2 * index + 1) + \
        self._query_range_sum(mid + 1, end, query_start, query_end, 2 * index + 2)

        def update_value(self, student_index, new_value):
        diff = new_value - self.tree[self.n + student_index - 1]
        self._update_value(0, self.n - 1, student_index, diff, 0)

        def _update_value(self, start, end, student_index, diff, index):
    if student_index < start or student_index > end:
        return

        self.tree[index] += diff
        if start != end:
        mid = (start + end) // 2
        self._update_value(start, mid, student_index, diff, 2 * index + 1)
        self._update_value(mid + 1, end, student_index, diff, 2 * index + 2)


# Get input from the user
arr = [int(x) for x in input("Enter the array elements separated by spaces: ").split()]

segment_tree = SegmentTree(arr)

print("Initial segment tree:", segment_tree.tree)

# Get query range from the user
query_start, query_end = [int(x) for x in input("Enter the query range (start end) separated by spaces: ").split()]

print("Sum of elements in range [{}, {}]: {}".format(query_start, query_end, segment_tree.query_range_sum(query_start, query_end)))

# Get student index and new value from the user
```

```
student_index, new_value = [int(x) for x in input("Enter the student index and new value separated by spaces: ").split()]

segment_tree.update_value(student_index, new_value)
print("Segment tree after updating value of student {}: {}".format(student_index, segment_tree.tree))

# Query range sum after update
print("Sum of elements in range [{}, {}]: {}".format(query_start, query_end, segment_tree.query_range_sum(query_start, query_end)))
```

OUTPUT

```
Output                                                    Clear
Enter the array elements separated by spaces: 10 20 30 40 50
Initial segment tree: [150, 60, 90, 30, 30, 40, 50, 10, 20, 0]
Enter the query range (start end) separated by spaces: 1 3
Sum of elements in range [1, 3]: 90
Enter the student index and new value separated by spaces: 2 25
Segment tree after updating value of student 2: [125, 35, 90, 30, 5, 40, 50, 10, 20
    , 0]
Sum of elements in range [1, 3]: 65

=== Code Execution Successful ===
```

Result:
Thus the Program was implemented successfully.

Experiment 6
Implement a geographic information system (GIS) for locating a city as node using quadtree

Aim:
To implement a geographic information system (GIS) for locating a city as node using quadtree

Algorithm:

1. Initialization (__init__):
   - Initialize the root node with the given boundary.
   - Set the capacity for each node.
2. Insert (insert):
   - Insert a city into the QuadTree. If the node exceeds its capacity, it subdivides and redistributes cities among the new children.
3. Subdivision (_subdivide):
   - Split a node into four children, each representing a quadrant of the original node's boundary. Redistribute the cities among the new children based on their coordinates.
4. Contains (_contains):
   - Check if a city's coordinates fall within a given boundary.
5. Query Location (query_location):
   - Find all cities within the boundary that contains the query point. This method checks each node recursively and collects cities from nodes that contain the query point.
6. Query Nearest City (query_nearest_city):

   - Find the city nearest to a given query point using Euclidean distance. This method examines each city in relevant nodes and keeps track of the city with the minimum distance to the query point.

Program:
```
import numpy as np

class QuadTreeNode:
        def __init__(self, boundary):
           self.boundary = boundary
        self.cities = []
        self.children = [None, None, None, None]
```

```
class QuadTree:
    def __init__(self, boundary, capacity):
    self.root = QuadTreeNode(boundary)
    self.capacity = capacity

    def insert(self, city):
    self._insert_recursive(self.root, city)

    def _insert_recursive(self, node, city):
    if len(node.cities) < self.capacity:
    node.cities.append(city)
    else:
    if node.children[0] is None:
        self._subdivide(node)
    for i in range(4):
        if self._contains(node.children[i].boundary, city):
      self._insert_recursive(node.children[i], city)

    def _subdivide(self, node):
    x, y, w, h = node.boundary
    half_w = w / 2
    half_h = h / 2
    node.children[0] = QuadTreeNode((x + half_w, y, half_w, half_h))
    node.children[1] = QuadTreeNode((x, y, half_w, half_h))
    node.children[2] = QuadTreeNode((x, y + half_h, half_w, half_h))
    node.children[3] = QuadTreeNode((x + half_w, y + half_h, half_w, half_h))
    for city in node.cities:
    for i in range(4):
        if self._contains(node.children[i].boundary, city):
        self._insert_recursive(node.children[i], city)
    node.cities = []

    def _contains(self, boundary, city):
    x, y, w, h = boundary
    return x <= city[0] < x + w and y <= city[1] < y + h

    def query_location(self, query_point):
    return self._query_location_recursive(self.root, query_point)

    def _query_location_recursive(self, node, query_point):
    if node is None:
    return []
    if not self._contains(node.boundary, query_point):
    return []
    if node.children[0] is None:
```

```
        return node.cities
        result = []
        for i in range(4):
        result.extend(self._query_location_recursive(node.children[i], query_point))
        return result

        def query_nearest_city(self, query_point):
        return self._query_nearest_city_recursive(self.root, query_point, float('inf'), None)

        def _query_nearest_city_recursive(self, node, query_point, min_dist, nearest_city):
        if node is None:
        return nearest_city
        if not self._contains(node.boundary, query_point):
        return nearest_city
        for city in node.cities:
        dist = np.linalg.norm(np.array(city) - np.array(query_point))
        if dist < min_dist:
                min_dist = dist
                nearest_city = city
        if node.children[0] is not None:
        for i in range(4):
                nearest_city  =  self._query_nearest_city_recursive(node.children[i],  query_point,
min_dist, nearest_city)
        return nearest_city

# User Input
boundary = tuple(map(int, input("Enter the map boundary (x, y, width, height): ").split()))
capacity = int(input("Enter the capacity of each quadtree node: "))
quadtree = QuadTree(boundary, capacity)

# Insert cities into the quadtree
num_cities = int(input("Enter the number of cities to insert: "))
cities = []
for _ in range(num_cities):
        city = tuple(map(int, input("Enter city coordinates (x, y): ").split()))
        cities.append(city)
        quadtree.insert(city)

# Query for city locations and nearest city
query_point = tuple(map(int, input("Enter the query point (x, y): ").split()))
print("Cities near query point:", quadtree.query_location(query_point))

nearest_city = quadtree.query_nearest_city(query_point)
print("Nearest city to query point:", nearest_city)
```

OUTPUT

```
Output

Enter the map boundary (x, y, width, height): 0 0 100 100
Enter the capacity of each quadtree node: 4
Enter the number of cities to insert: 5
Enter city coordinates (x, y): 20 30
Enter city coordinates (x, y): 50 70
Enter city coordinates (x, y): 80 50
Enter city coordinates (x, y): 10 90
Enter city coordinates (x, y): 40 10
Enter the query point (x, y): 25 35
Cities near query point: [(20, 30), (40, 10)]
Nearest city to query point: (20, 30)

=== Code Execution Successful ===
```

Result:
Thus the program was implemented successfully.